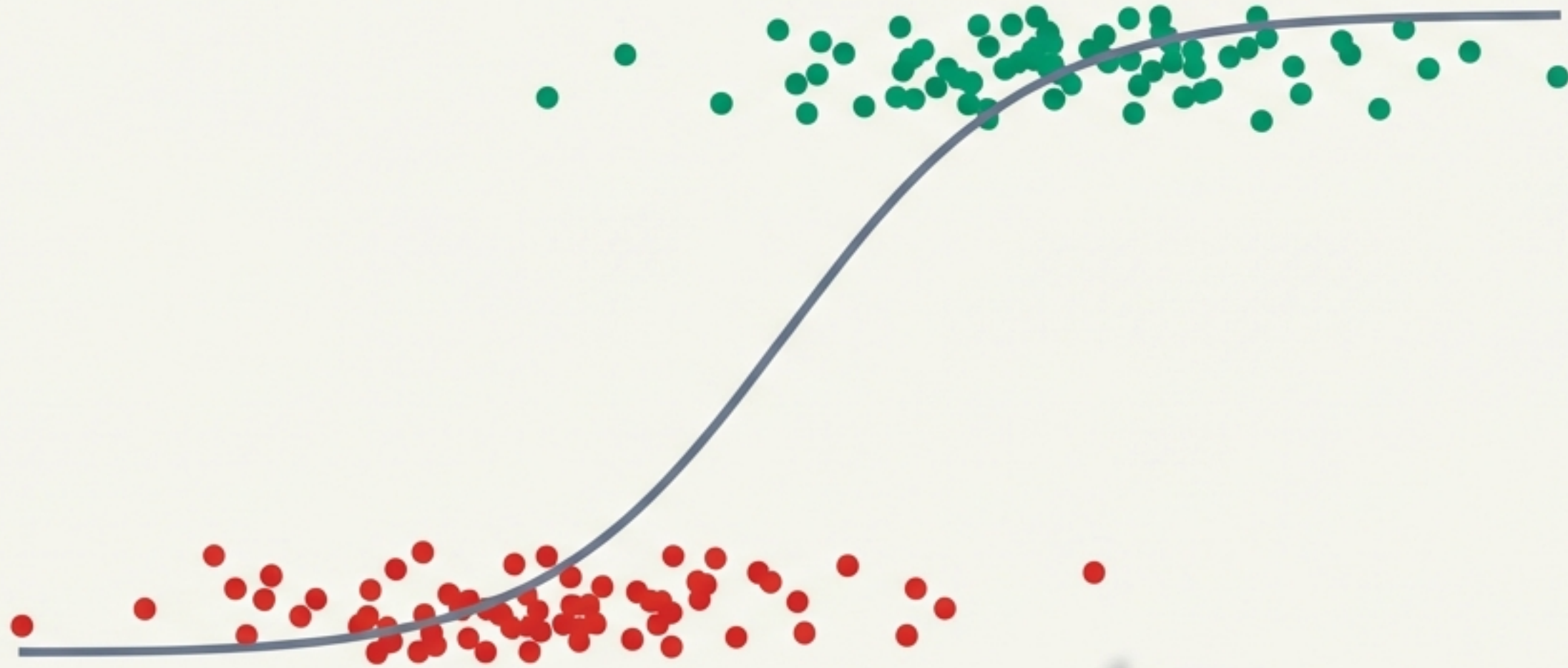


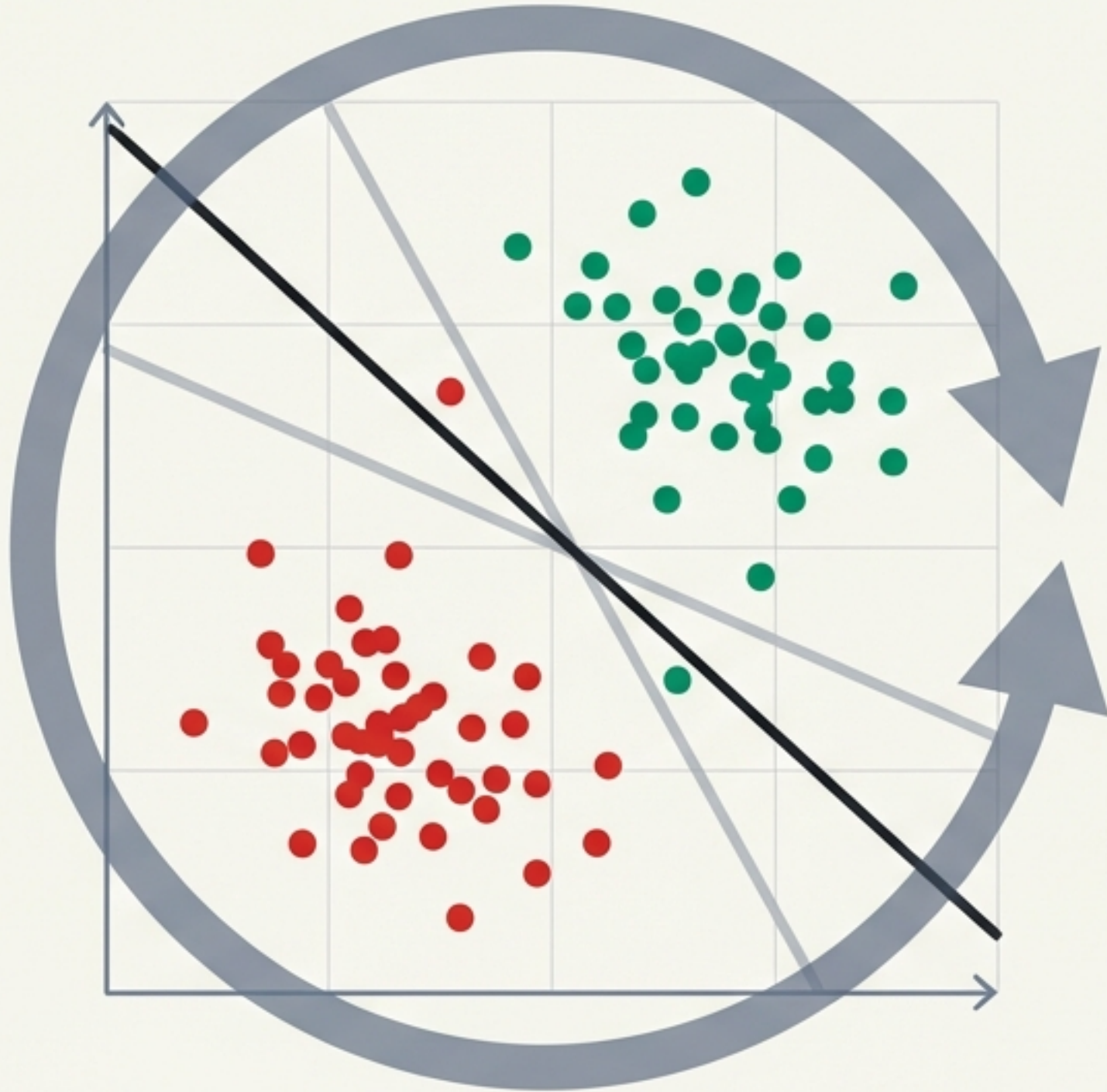
The Evolution of Log Loss

Deriving Binary Cross Entropy from first principles.



$$L = -\frac{1}{N} \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

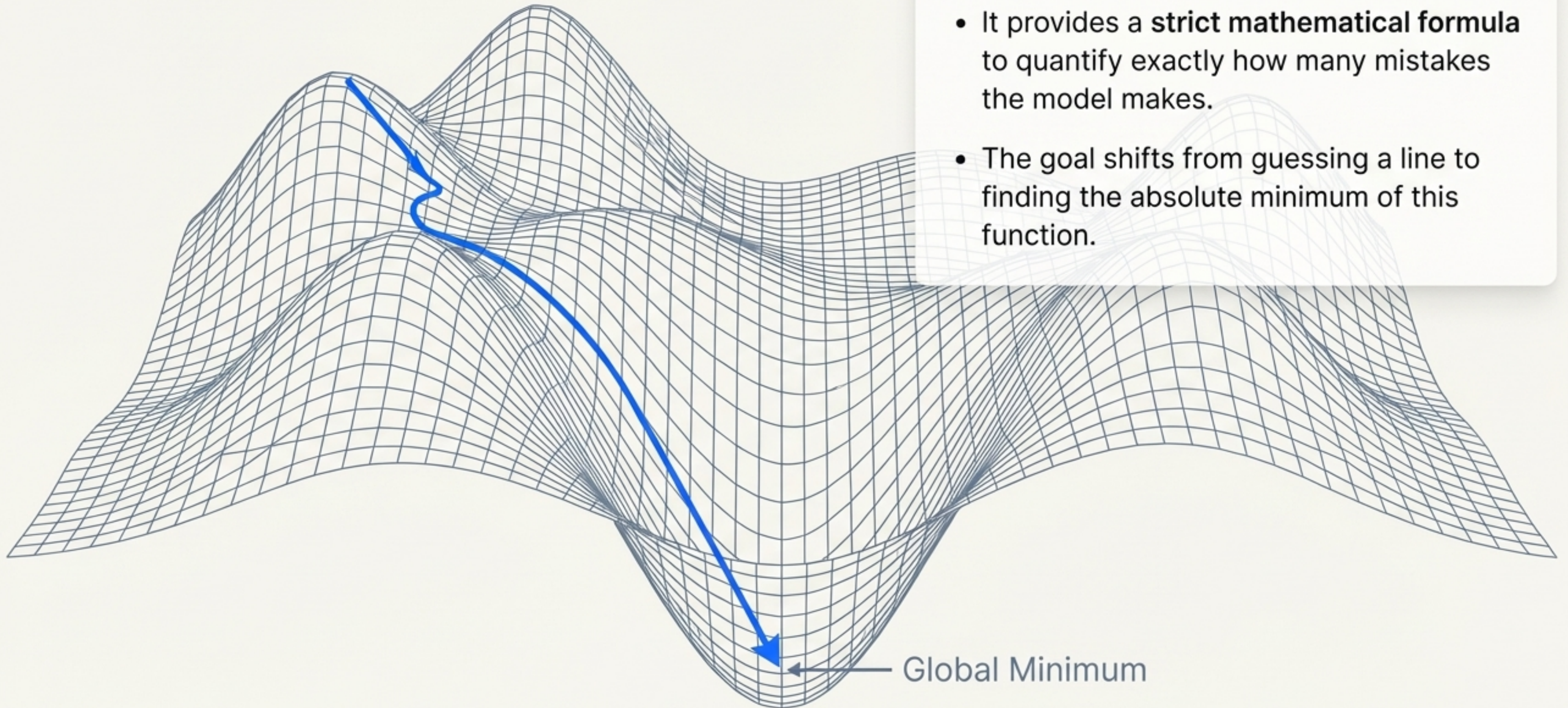
The Flaw in the Perceptron



- Randomly adjusting decision lines based on misclassified points is a guessing game.
- We can loop 1,000 times, but we cannot guarantee the result is the absolute optimum solution.
- We need mathematical conviction, not just a 'good enough' line.

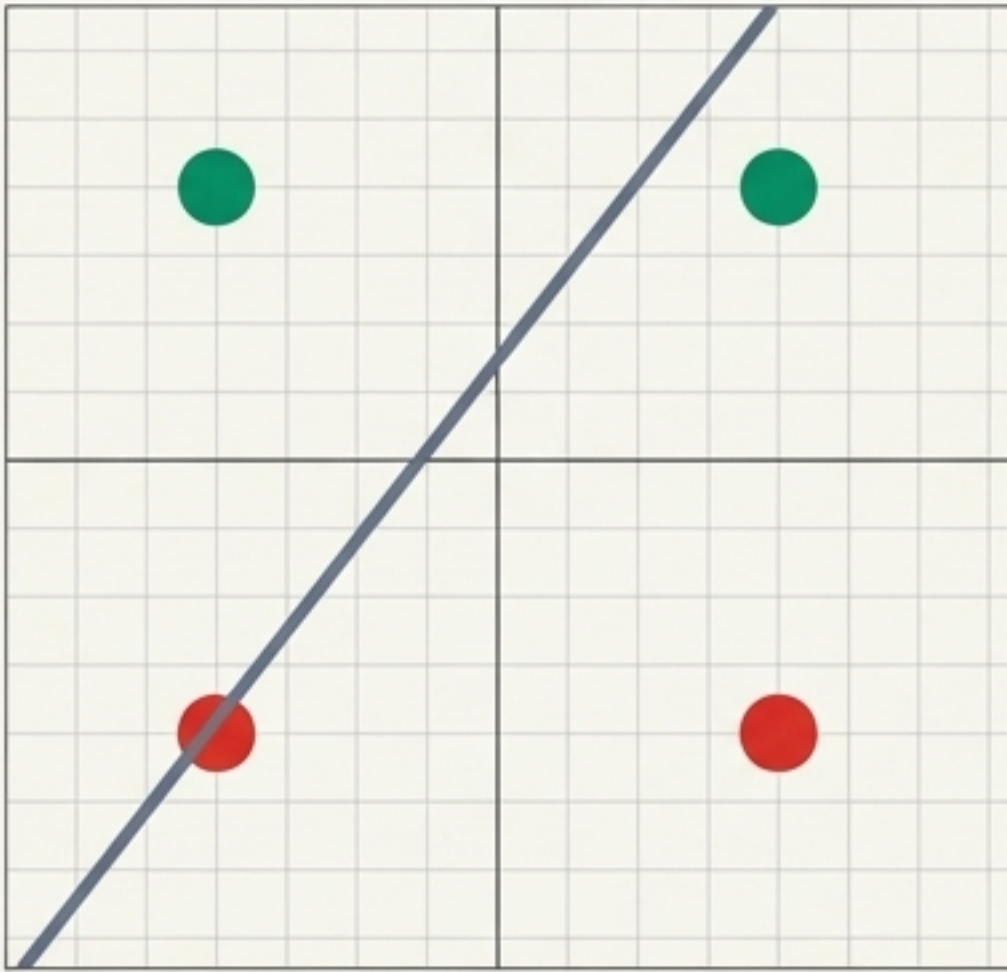
The Machine Learning Paradigm

- True machine learning relies on a **Loss Function** (or Error Function).
- It provides a **strict mathematical formula** to quantify exactly how many mistakes the model makes.
- The goal shifts from guessing a line to finding the absolute minimum of this function.

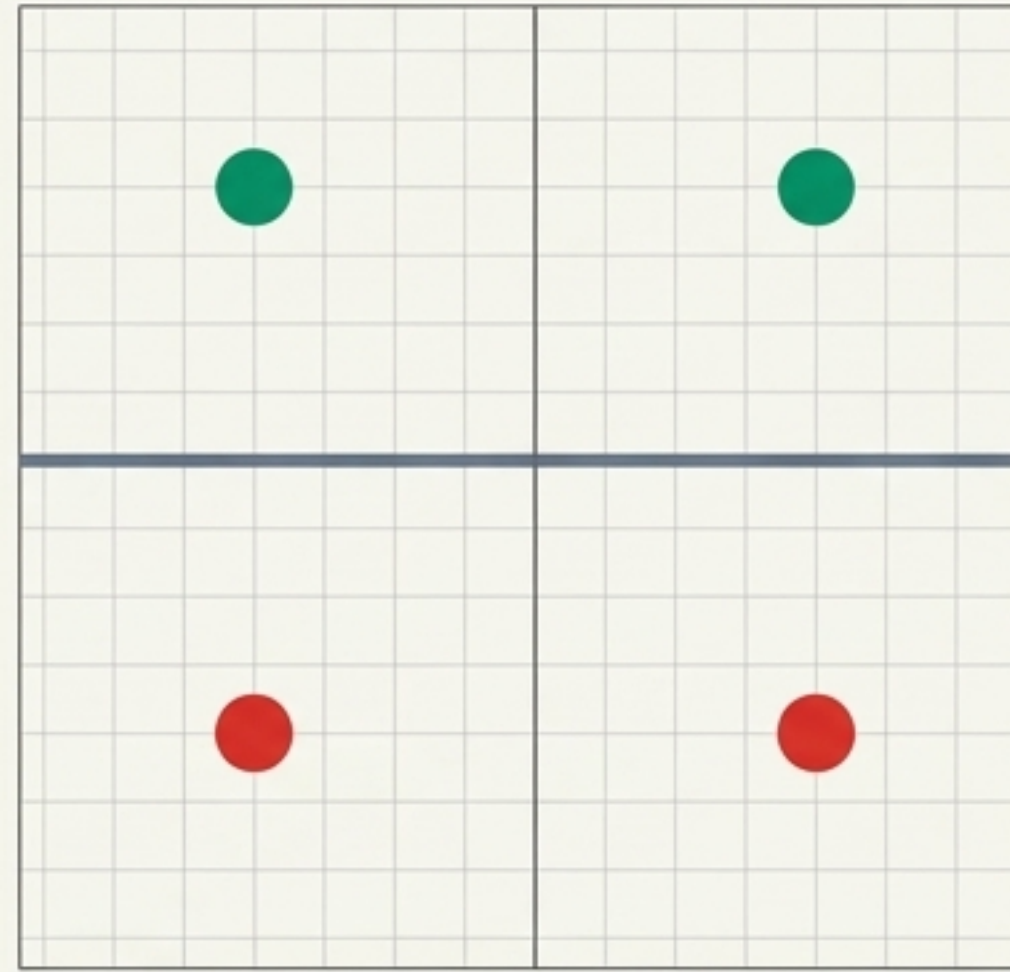


Visual Intuition Requires Mathematical Proof

Model 1



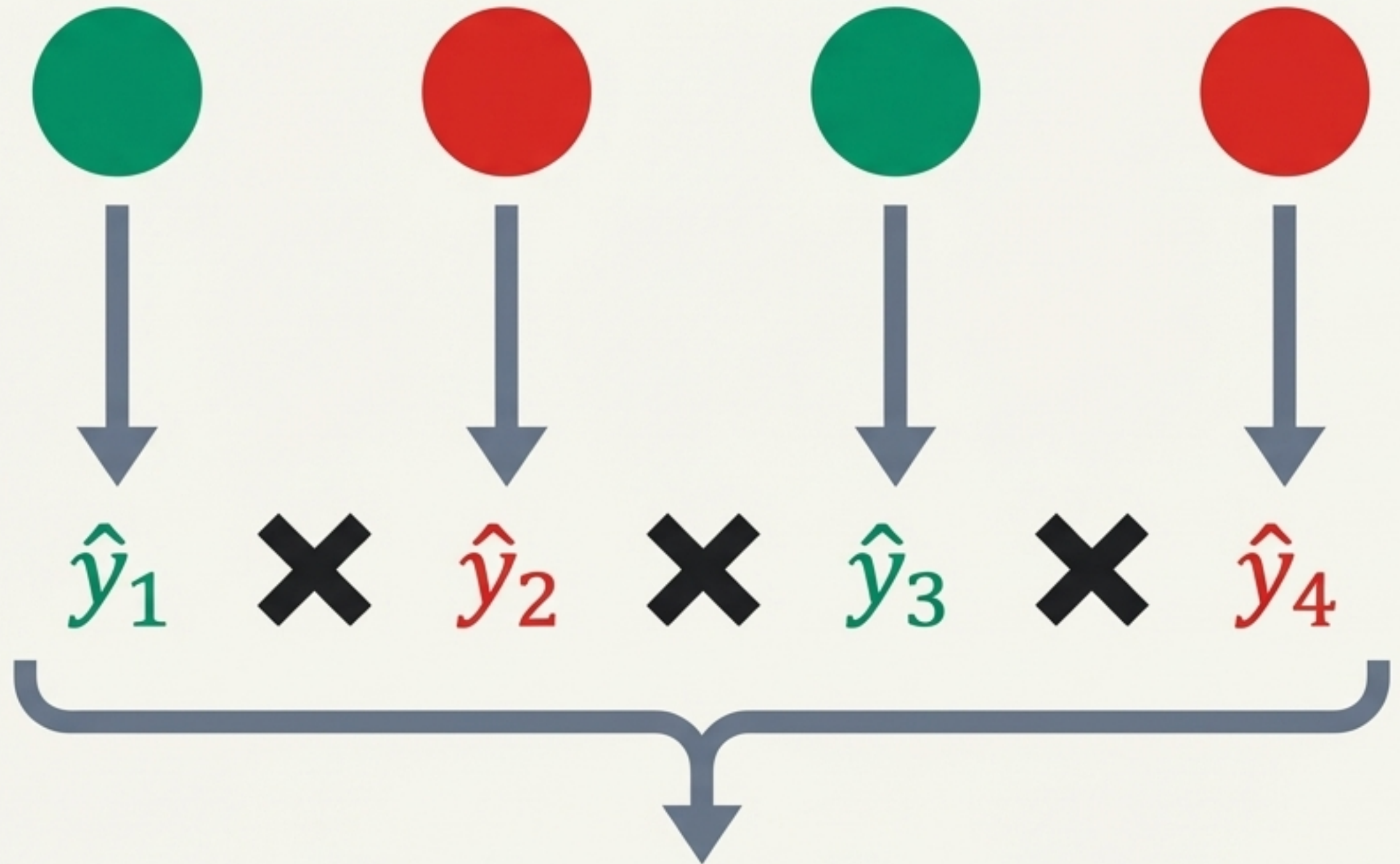
Model 2



- To a human, Model 2 is obviously better at classifying the classes.
- But how do we teach a machine to evaluate this comparison automatically?

Enter Maximum Likelihood

- The logic: Multiply the predicted probabilities of the correct class for every single data point.
- A higher final product indicates a better model.
- *Note: Only use the probability of the true colour. If a point is Red, use the probability of it being Red.*



Maximum Likelihood

Calculating the Likelihood

Model 1 Calculation

$$\begin{aligned} &0.7 \times 0.4 \text{ \#DC2626} \times \\ &\times 0.4 \text{ \#059669} \times \\ &\times 0.8 \text{ \#DC2626} = \\ &= 0.0896 \end{aligned}$$

Model 2 Calculation

$$\begin{aligned} &0.7 \times 0.6 \text{ \#059669} \times \\ &\times 0.4 \text{ \#DC2626} \times \\ &\times 0.7 \text{ \#DC266} = \\ &= 0.1176 \end{aligned}$$

Mathematically confirmed: $0.1176 > 0.0896$.
Model 2 has the higher Maximum Likelihood.

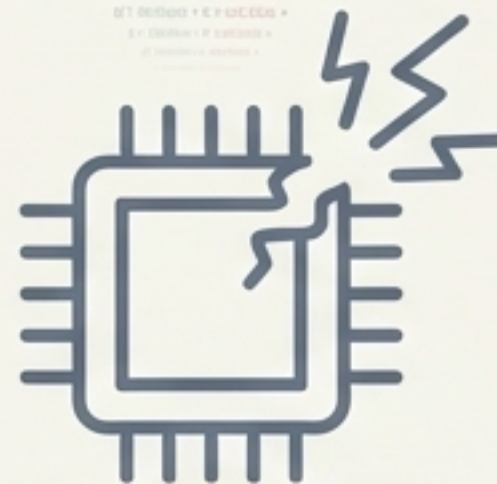
The Computational Roadblock

- Our dataset only had 4 points. Real-world data has 10,000+ points.
- Multiplying thousands of probabilities (numbers between 0 and 1) creates infinitesimally small numbers.
- This causes computational underflow. The computer simply rounds to zero, destroying our comparison.

0 . 000000000000000000000000...

0.7 #859669 × 0.4 #DC2626 × 0.9 #859669 × 0.2 #DC2626 × 0.6 #859669 × 0.8 #DC2626 ×
0.7 #859669 × 8.4 #9C2626 × 0.9 #859669 × 6.2 #DC2626 × 0.6 #059669 × 0.8 #DC2626 ×
0.7 #059669 × 8.4 #0315 × 0.9 #0C2626 × 0.2 #DC2626 × 0.6 #059669 × 0.8 #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.9 #059669 × 0.6 #059669 × 0.8 #DC2626 ×
0.7 #059669 × 0.4 A9C2626 × 0.9 A059669 × 0.6 #059669 × 0.5 #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.7 A815 × 0.2 8059669 × 0.8 80C2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.9 #05325 #859669 × 0.8 #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.25 #059669 × 0.8 #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.6 #059669 × 0.8 #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.6 #059669 × 0.8 #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.6 #059669 × #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.6 #059669 #DC2626 ×
0.7 #059669 × 0.4 #DC2636 × 0.9 9:08 #DC2626 ×
0.7 #059669 × 0.4 #DC2626 × 0.8 #DC2626 ×
0.7 #859669 × 0.4 #9C2626 × 0.8 #002626 ×
0.7 #859669 × 0.4 #8C2626 × 0.8 #8C2636 ×
0.7 #859669 × 6.4 #DE2626 × 6.5 #BE2526 +
0.7 #859669 × 5.4 #DE2E26 × 6.4 #DC2626 ×
0.1 10A91M → 0.8 #BCB100 → 5.0 #DE2636 ×
0.7 #859669 × 0.8 #DC2626 × 2.000000 ×
0.7 10A91M → 6.4 #DC2626 ×
0.1 #859669 × 0.8 #DC2626 ×
0.1 #859669 × 0.8 #DC2626 ×

✓



0 . 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ...

The Logarithmic Rescue

By calculating the Logarithm of the Maximum Likelihood, we transform fragile multiplication into stable addition.

New points are simply added to the running total.

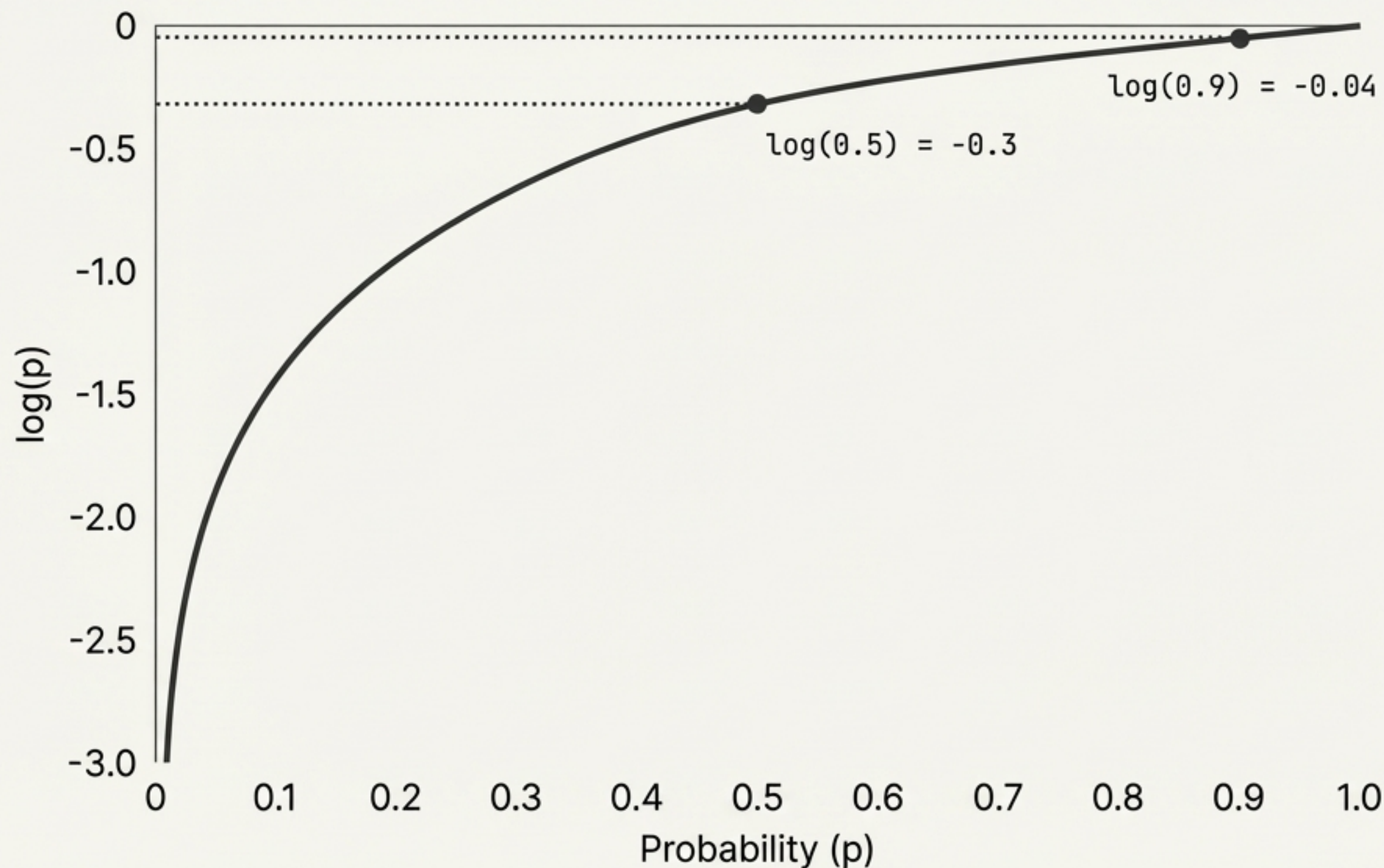
$$p_1 \times p_2 \times p_3 \times p_4$$

$$\log(ab) = \log(a) + \log(b)$$

Apply Logarithm

$$\log(p_1) + \log(p_2) + \log(p_3) + \log(p_4)$$

The Negative Sign Anomaly



Because probabilities always sit between 0 and 1, their logarithms are strictly negative.

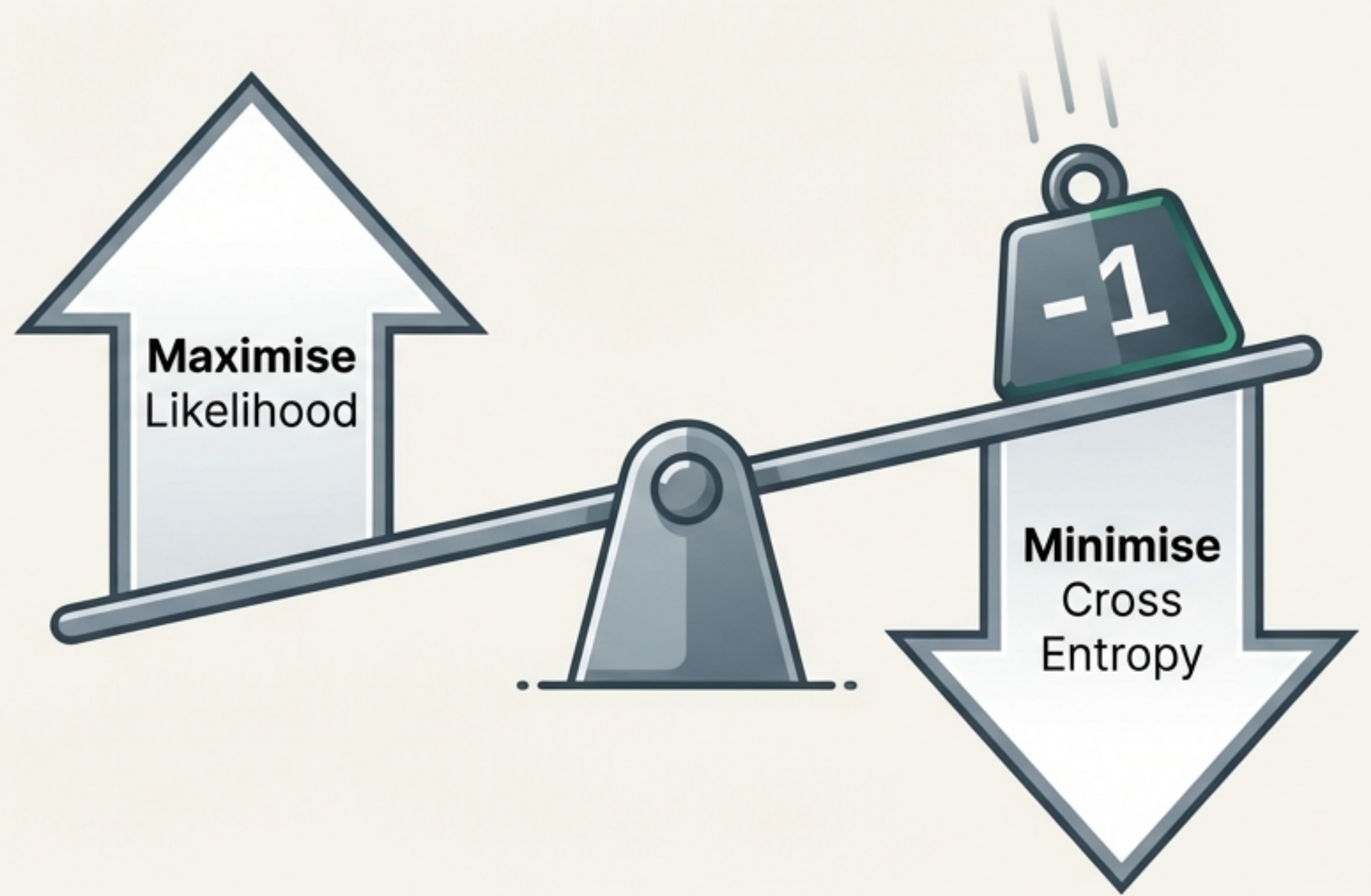
The Fix: We multiply the entire sum by -1 to force the metric to be a positive, readable number.

Maximise to Minimise

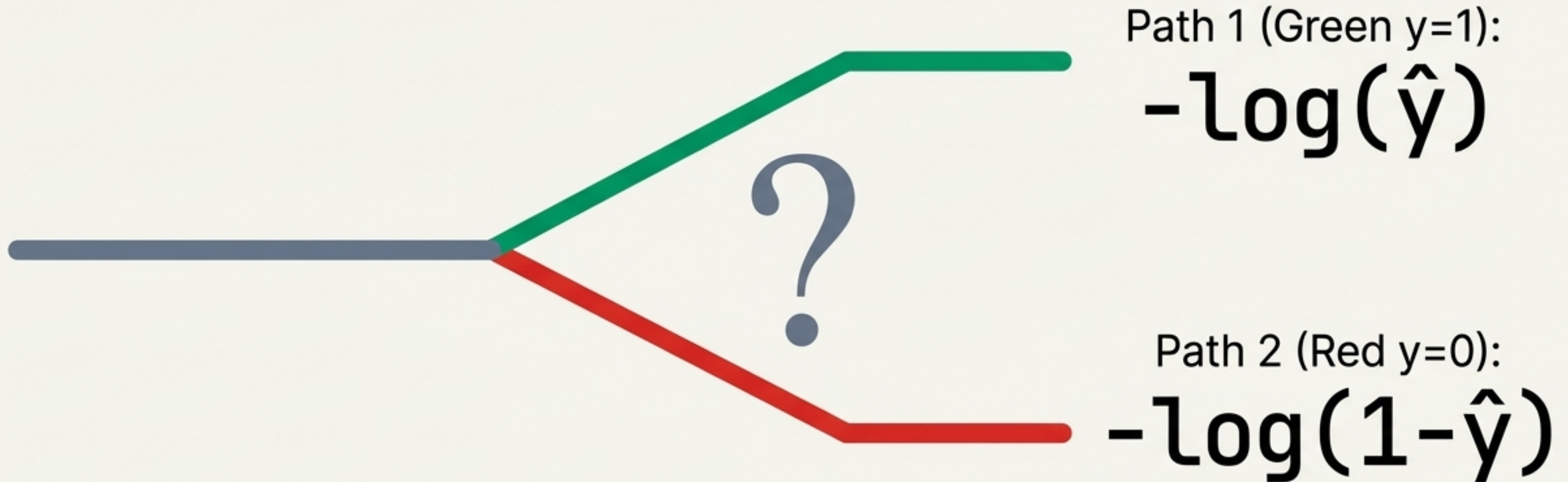
Adding the **negative sign** flips our core **objective**.

Smaller probabilities yield **larger negative logs**. When inverted, a larger error yields a larger positive penalty.

We no longer seek to maximise a score; we seek to **minimise** the **Cross Entropy**. The lowest score is the best model.



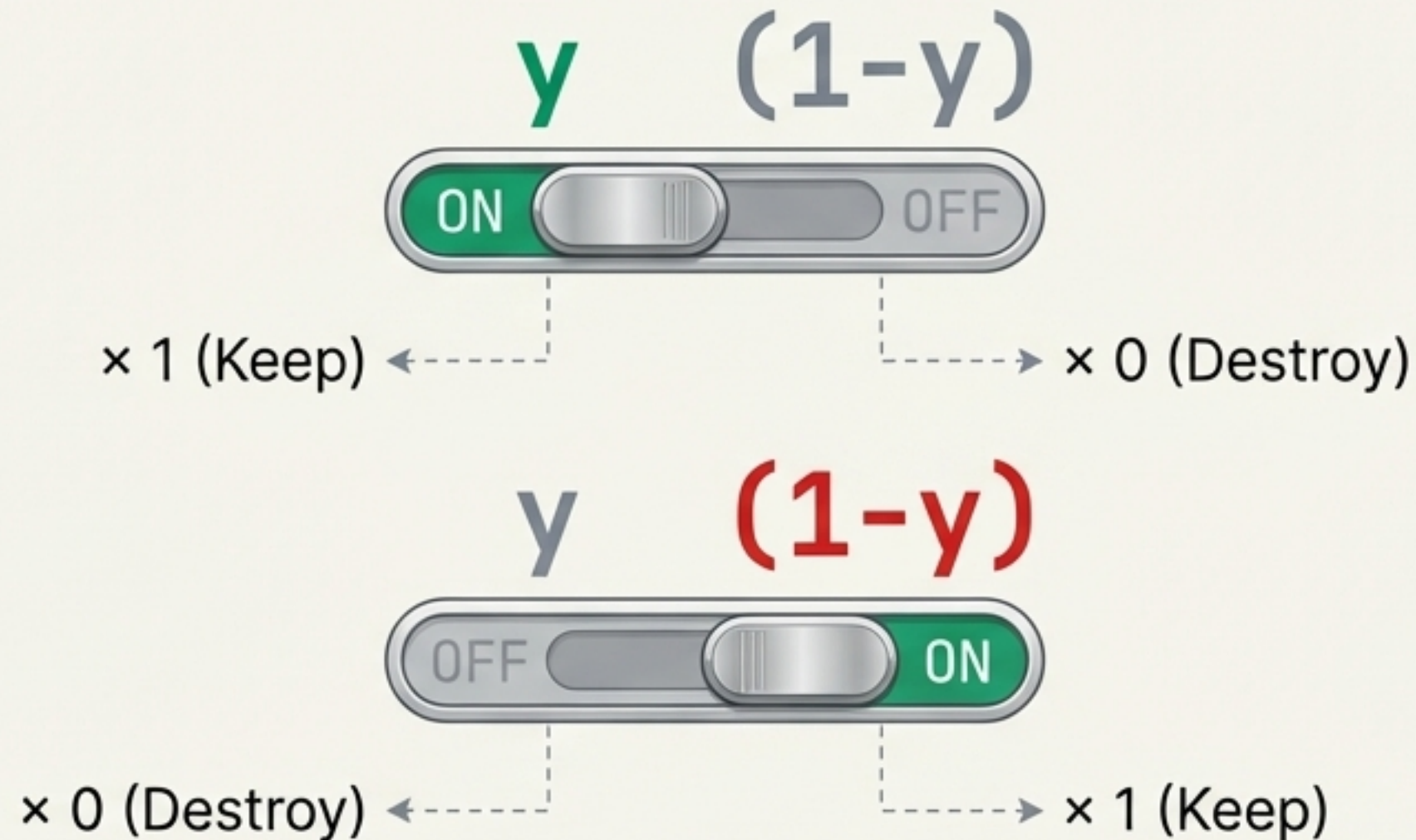
The Need for a Generalised Formula



- We cannot manually hardcode a different equation for Green points and Red points in our loop.
- We need a single, unified mathematical equation that works for both scenarios automatically.

Building the 'Toggle' Mechanism

- We introduce y (the true label) as a multiplier.
- This acts as an automatic switch, instantly zeroing out the half of the formula we do not need for any given data point.



Testing the Formula

The generalized format handles both classes flawlessly without manual intervention.

Green Point ($y=1$):

$$1 \cdot \log(\hat{y}) + \cancel{0 \cdot \log(1 - \hat{y})}$$

Red Point ($y=0$):

$$\cancel{0 \cdot \log(\hat{y})} + 1 \cdot \log(1 - \hat{y})$$

Binary Cross Entropy (Log Loss)

Averaging the penalty and keeping it positive.

The penalty applied if the true class is Green ($y=1$).

$$L = -\frac{1}{N} \sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

Adding up the stable errors of every data point.

The penalty applied if the true class is Red ($y=0$).

We have successfully mathematically quantified our error.

The Next Frontier: Optimisation

We have built our Loss Function, but unlike Linear Regression, this formula has no closed-form solution.

We cannot calculate the minimum directly.

To update our weights and find the optimum model, we must apply **Gradient Descent**.

